Лабораторная работа №11
Разработка web-API для доступа к данным

1**Цель работы**

1.1 Научиться выполнять разработку web-API для доступа к БД.

2**Литература**

2.1 ASP.NET Core MVC | Полное руководство. metanit.com – Текст : электронный // metanit.com, 2025. – URL: https://metanit.com/sharp/aspnet6/ – гл.11-12.

3**Подготовка к работе**

3.1 Повторить теоретический материал (см.п.2).

3.2 Изучить описание практической работы.

4**Основное оборудование**

4.1 Персональный компьютер.

5**Задание**

5.1 Создание структуры проектов

5.1.1 Создать проект «Веб-API ASP.NET Core (Майкрософт)». В окне «Дополнительные сведения» выбрать платформу .NET 10 и поставить флажки «Включить поддержку OpenAPI» и «Использовать контроллеры».

5.1.2 Добавить в проект поддержку Scalar для тестирования web-API (установитть пакет Scalar.AspNetCore).

5.1.3 Реализовать запуск браузера и открытие страницы со Scalar при старте приложения. Для этого:

- добавить в Program.cs в блок if вызов метода app.MapScalarApiReference();
- в файле Properties/launchSettings.json указать:

"launchBrowser": true,
"launchUrl": "scalar",

- сохранить файл launchSettings.json.

5.1.4 Добавить в проект пакеты для работы с БД (Microsoft.EntityFrameworkCore.SqlServer и Microsoft.EntityFrameworkCore.Tools). Сгенерировать контекст данных и модели данных для таблиц предметной области «Кинотеатр», используя команду Scaffold-DbContext (без перевода на новую строку и заменив выделенные цветом данные):

```
Scaffold-DbContext "Data Source=mssql; Initial Catalog=isppXXNN; Persist Security Info=True;User ID=isppXXNN; Password=XXNN; TrustServerCertificate=True" Microsoft.EntityFrameworkCore.SqlServer -Context ИмяКонтекстаContext -ContextDir ИмяДиректории -OutputDir Models -Force
```

Чтобы не включать в JSON-ответ объёмные и навигационные данные, над свойствами в модели можно указать аннотацию [JsonIgnore], например, над постером у фильма и над списком фильмов в жанрах.

5.1.5 Сгенерировать в веб-API контроллер для класса Фильм. Для этого в контекстном меню директории Controllers выбрать Добавить – Контроллер – API – Контроллер API с действиями, использующий EF. В окне выбрать класс модели фильм и класс контекста БД.

При генерации контроллера в Program.cs в сервисы проекта автоматически добавляется контекст: builder.Services.AddDbContext<ИмяКонтекста>(…);

Если контроллер не генерируется автоматически:
- добавить в проект пустой контроллер,
- в основной конструктор добавить параметр контекста,
- добавить метод Get для получения набора данных,
- добавить в Program.cs строку builder.Services.AddDbContext<ИмяКонтекста>();

```
[Route("api/[controller]")]
[ApiController]
Ссылок: 0
public class MoviesController(CinemaContext context) : ControllerBase
{
    // GET: api/<MoviesController>
    [HttpGet]
    Ссылок: 0
    public IEnumerable<Movie> Get()
    {
        return context.Movies.ToList();
    }
}
```

5.2 Создание GET-методов с параметрами запроса

Добавить в приложение методы:

- /films/pages?page={page}&sortBy={sortBy} для постраничного вывода информации о фильмах (размер страницы 3). Включить в метод возможность сортировки по названию, году выхода по возрастанию и году выхода по убыванию,
- /films/filter?year={year}&title={title} для получения списка фильмов указанного года и/или содержащих указанный текст в названии.

Для отличия методов кроме разных названий указать разные аннотации:
- [HttpGet("pages")]
- [HttpGet("filter")]

Проверить работу созданных методов в браузере.

5.3 Создание GET-методов с параметрами пути

Добавить в приложение методы:

- /films/{id}/genres для получения списка жанров фильма с указанным в пути id,
- /films/{id}/sessions для получения списка будущих сеансов фильма с указанным в пути id.

Проверить работу созданных методов в браузере.

5.4 Создание GET-методов с составными параметрами запроса

Добавить в приложение метод /films/search

Добавить в созданный метод необязательные параметры:

- year={minYear}-{maxYear} для получения списка фильмов с годом из указанного диапазона (включительно), данные разделены дефисом,
- genres={genre1},{genre2},… для получения списка фильмов, у которых жанр входит в указанный в списке через запятую.

Проверить работу созданного метода браузере.

5.5 Создание GET-методов, возвращающих DTO

Добавить в приложение класс FilmDto {Id, Title, SalesProfit, TicketsCount}.

SalesProfit – сумма, полученная за продажу билетов на фильм.

Добавить в приложение методы:

- /films/statistics для получения информации о количестве проданных билетов и прибыли, полученной за продажу билетов (данные вернуть в формате FilmDto).
- /films/statistics/{id} для получения информации о количестве проданных билетов и прибыли, полученной за продажу билетов с указанным в пути id (данные вернуть в формате FilmDto). Проверить работу созданных методов в браузере.

6Порядок выполнения работы

6.1 Выполнить все задания из п.5.

6.2 Ответить на контрольные вопросы.

7Содержание отчета

7.1 Титульный лист

7.2 Цель работы

7.3 Ответы на контрольные вопросы

7.4 Вывод

8Контрольные вопросы

8.1 Что такое REST-запрос?

8.2 Что такое RESTful?

8.3 Для чего используется метод GET?

8.4 Для чего используется метод POST?

8.5 Для чего используется метод PUT?

8.6 Для чего используется метод DELETE?